TEAM G1 · TECHNICAL GUIDE · GNU RADIO 3.10

How to Add a Block to GNU Radio, from First Principles

From "what is a block, physically?" to a tested Gain block, a C++ out-of-tree module, the rules that prevent freezes, and the path to an upstream pull request.

What is different from the earlier gnu_block.txt notes

- Starts from why blocks look the way they do (buffers, scheduler), so the code makes sense instead of being copied.
- Every Python example in §4 was **run** on GNU Radio 3.10.12. The test output shown is real. The code is in the team repository: `M2_g1_gsoc_wifisensing/Code/gain_block_tutorial/`.
- §5 shows the real file tree that `gr_modtool` generated for a C++ block.
- §6 connects block-writing rules to the freezes we measured in M2 (#7997).
- §7 explains when a block is, and is not, an upstream contribution.

1. What a block is, physically

Think of a factory line. Each worker (a **block**) takes items from a tray on their left, does one job, and puts results on a tray on their right. The trays are **buffers**, fixed-size areas of memory. A supervisor (the **scheduler**) keeps checking: "Does this worker have enough items on the left tray, and enough empty space on the right tray? If yes, let them work."

- An **item** is one unit on the tray: a float, a complex number, a byte, or a vector of 64 complex numbers (one OFDM symbol).
- In GNU Radio 3.x, every block runs in **its own thread** ("thread-per-block scheduler").
- The scheduler calls your block's `work()` or `general_work()` with a chunk of items. The chunk size changes from call to call. **Your block must give the right answer for any chunk size.** That is why tests use long streams with many calls (see test_003 in §4).
- Your function returns **how many items it produced**, and for general blocks it also says how many it **consumed**. The scheduler uses these numbers to advance the trays.

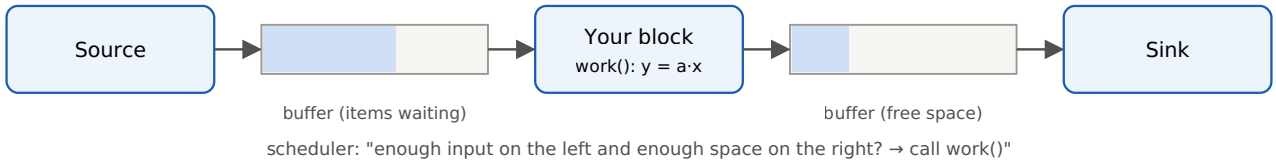


Figure 1. Blocks, buffers and the scheduler.

2. The block types, and how to choose one

Start with one question: **how many output items do I make per input item?**

Base class	Input : output	You write	Example
`sync_block`	1 : 1	`work()`	Gain, multiply, add noise
`decim_block`	N : 1	`work()`	Keep 1 sample in N
`interp_block`	1 : N	`work()`	Repeat each sample N times
`basic_block` (general block)	anything, variable	`forecast()` + `general_work()` + `consume_each()`	Packet detector, resampler, Header/ Payload Demux
`tagged_stream_block`	one whole packet at a time	`work()` + length tag	OFDM carrier allocator, CRC32
`hier_block2`	a group of blocks shown as one	only `connect()` calls	`ofdm_tx`, `ofdm_rx`, Channel Model

Rule of thumb: use the simplest type that fits. A `sync_block` cannot get the item bookkeeping wrong, because the runtime does it for you. General blocks are powerful but are where most deadlocks come from (§6).

Besides streams, blocks can have **message ports** (asynchronous, e.g. "header decoded: length = 50") and read or write **stream tags** (metadata pinned to one sample, e.g. "packet starts here"). `ofdm_rx` uses both.

3. Three ways to make a block

Way	Speed to start	Good for	Not good for
Embedded Python Block in GRC	minutes	learning, quick experiments	sharing, testing, performance
Python block in a module (plain Python class or `gr_modtool add -l python`)	an hour	prototypes with unit tests (our §4)	high sample rates
C++ block in an out-of-tree (OOT) module (`gr_modtool add -l cpp`)	an afternoon	performance; the form required upstream	quick experiments

GNU Radio's CONTRIBUTING.md: "all performance-critical code should be written in C++. The same goes for all blocks, since there are use cases for C++-only GNU Radio." Prototype in Python, then contribute blocks in C++.

4. Worked example: a Gain block y[n] = a·x[n] in Python

Example: a = 2, x = [1, 2, 3, 4] → y = [2, 4, 6, 8]. It is a `sync_block` because there is exactly one output per input.

4.1 The block (`gain_block.py`)

```
import numpy as np
from gnuradio import gr

class gain_ff(gr.sync_block):              # 1 output item per input item
    """Multiply every float sample by a constant gain a."""

    def __init__(self, gain=1.0):
        gr.sync_block.__init__(
            self,
            name="gain_ff",
            in_sig=[np.float32],            # one input port carrying float32
            out_sig=[np.float32],           # one output port carrying float32
        )
        self.gain = float(gain)

    def set_gain(self, gain):               # callback: GRC can change it while running
        self.gain = float(gain)

    def work(self, input_items, output_items):
        x = input_items[0]                  # numpy view into the input buffer
        y = output_items[0]                 # numpy view into the output buffer
        y[:] = self.gain * x                # the whole algorithm
        return len(y)                       # "I produced this many items"
```

Line by line, the parts that matter:

- `in_sig` / `out_sig` declare the item type. The runtime uses them to size buffers and to check connections.
- `input_items[0]` is a NumPy array that points into the buffer. No copy is made. Write with `y[:] = …`; `y = …` would only rebind the Python name and output nothing.
- `len(y)` is the number of items the scheduler asked for. For a `sync_block` the input has the same length.

4.2 The test (`test_gain_block.py`), in GNU Radio's own test style

```
class qa_gain_ff(gr_unittest.TestCase):
    def setUp(self):    self.tb = gr.top_block()
    def run_block(self, data, gain):
        src, dut, snk = blocks.vector_source_f(data, False), gain_ff(gain),
blocks.vector_sink_f()
        self.tb.connect(src, dut, snk); self.tb.run(); return snk.data()

    def test_001_known_values(self):         # the example from the spec
        self.assertFloatTuplesAlmostEqual(self.run_block([1, 2, 3, 4], 2.0), [2, 4, 6, 8],
6)
    def test_002_edge_cases(self):            # gains -1, 0, 0.5; negative inputs
        ...
    def test_003_long_stream_matches_builtin(self):
        # 100 000 random samples -> many work() calls with different chunk sizes.
        # Reference ("oracle"): GNU Radio's own blocks.multiply_const_ff
        ...
    def test_004_empty_input(self):           # nothing in -> nothing out, and the
flowgraph ends
        self.assertEqual(len(self.run_block([], 2.0)), 0)
```

```
$ python test_gain_block.py
....
Ran 4 tests in 0.056s
OK
```

> **Why these four tests**
>
> **Known values** check the formula. **Edge cases** check the boundaries (zero, negative). The **long stream against a trusted reference** catches chunk-boundary bugs that short tests never trigger. **Empty input** checks that the flowgraph terminates instead of hanging. These four ideas work for any block you write.

4.3 See it run (`gain_demo_flowgraph.py`)

Signal Source (50 Hz sine, 1 kHz sampling) → Head (10 samples) → Gain(2) → Vector Sink. Real output:

```
n      x[n]      y[n] = 2 x[n]
0    0.0000       0.0000
1    0.3090       0.6180
2    0.5878       1.1756
3    0.8090       1.6180
4    0.9511       1.9021
5    1.0000       2.0000
```

4.4 Same block in GRC (Embedded Python Block)

1. Open `gnuradio-companion`, drag in Embedded Python Block, and double-click it to open its editor.
2. Replace the template class with the class from §4.1. The class must be named `blk` in an Embedded Python Block, and every `__init__` argument needs a default value. GRC turns the arguments into block parameters.
3. Connect: Signal Source (float) → your block → QT GUI Time Sink. Add a second Time Sink input directly from the source to compare. Run: the amplitude doubles.

5. The C++ way: an out-of-tree module with gr_modtool

5.1 Commands

```
gr_modtool newmod wifisense                 # creates gr-wifisense/
cd gr-wifisense
gr_modtool add -t sync -l cpp --argument-list "float gain=1.0" --add-cpp-qa gain_ff
#   asks: copyright holder?  -> your team name
#   asks: add Python QA code? -> y (recommended)
```

5.2 What it generated (real output, GNU Radio 3.10.12)

```
CMakeLists.txt                          top-level build script
include/gnuradio/wifisense/gain_ff.h    PUBLIC interface: make(float gain), set_gain()
lib/gain_ff_impl.h                      PRIVATE implementation class
lib/gain_ff_impl.cc                     constructor + work()  <- your algorithm goes here
lib/qa_gain_ff.cc                       C++ unit test
python/wifisense/bindings/gain_ff_python.cc   pybind11 glue: makes it importable in Python
python/wifisense/bindings/docstrings/...      docstrings for Python
grc/wifisense_gain_ff.block.yml         how the block looks in GRC (must be edited!)
python/wifisense/__init__.py
MANIFEST.yml, .clang-format, .conda/recipe/   metadata, formatting, packaging
```

This split of **public interface / private implementation / bindings / GRC description / tests** is how every C++ block in GNU Radio itself is laid out. Once you know it, you can find your way around `gr-digital`.

5.3 Fill in the algorithm (`lib/gain_ff_impl.cc`)

The generated `work()` contains a placeholder and a `#pragma message` reminder. Replace them:

```
gain_ff_impl::gain_ff_impl(float gain)
  : gr::sync_block("gain_ff",
        gr::io_signature::make(1, 1, sizeof(input_type)),
        gr::io_signature::make(1, 1, sizeof(output_type))),
    d_gain(gain)                                // store the parameter (declare float
d_gain in _impl.h)
{}

int gain_ff_impl::work(int noutput_items,
                       gr_vector_const_void_star& input_items,
                       gr_vector_void_star& output_items)
{
    auto in  = static_cast<const input_type*>(input_items[0]);
    auto out = static_cast<output_type*>(output_items[0]);
    for (int i = 0; i < noutput_items; i++)
        out[i] = d_gain * in[i];                // y[n] = a x[n]
    // faster: volk_32f_s32f_multiply_32f(out, in, d_gain, noutput_items);
    return noutput_items;
}
```

Also: set `input_type` / `output_type` to `float` at the top of the file; add `float d_gain;` and `void set_gain(float g) override { d_gain = g; }` to `gain_ff_impl.h`; declare `virtual void set_gain(float g) = 0;` in the public header; run `gr_modtool bind gain_ff` to regenerate the Python bindings after changing the header.

5.4 Make it appear in GRC (`grc/wifisense_gain_ff.block.yml`)

```
parameters:
- id: gain
  label: Gain
  dtype: float
  default: '1.0'
inputs:
- label: in
  dtype: float
outputs:
- label: out
  dtype: float
templates:
  imports: from gnuradio import wifisense
  make: wifisense.gain_ff(${gain})
  callbacks:
  - set_gain(${gain})
```

5.5 Build, install, test

```
mkdir build && cd build
cmake ..                    # with conda: cmake -DCMAKE_INSTALL_PREFIX=$CONDA_PREFIX ..
make -j4
make test                   # runs qa_gain_ff (C++) and the Python QA test
sudo make install && sudo ldconfig      # with conda: make install (no sudo)
```

Build requirements: a C++17 compiler, CMake, pybind11 and GNU Radio development headers. On Ubuntu: `sudo apt install gnuradio-dev cmake g++ pybind11-dev`. In a conda/micromamba environment: `micromamba install -c conda-forge cmake cxx-compiler pybind11 gnuradio`. We generated the skeleton

in §5.2 with GNU Radio 3.10.12 but did not compile it in the M2 environment (no compiler installed), so do this step on your own machine and record any problems in the dev diary.

6. The rules that prevent freezes (lessons from M2 and #7997)

Issue #7997 describes a custom `basic_block` with `general_work()` and `consume_each()` on vector ports, followed by a freeze of the whole flowgraph. In M2 we showed a mechanism that produces exactly that freeze. The runtime check is in `block_executor.cc`:

```
int best_n = round_down(out_buf->bufsize() / 2, output_multiple);
if (best_n < min_noutput_items)
    throw std::runtime_error("Buffer too small for min_noutput_items");
```

If any block asks for more output space than **half its buffer**, its thread dies. Everyone else waits forever, and back-pressure stops even the source. No exception reaches your Python code.

#	Rule for general / tagged-stream blocks	What happens if you break it
1	`forecast()` must request exactly the input you need for `noutput_items`, never more than the input buffer can ever hold.	The block waits for input that cannot arrive: deadlock.
2	Always `consume_each()` what you used, even when you produce nothing.	The input buffer never empties: upstream stalls.
3	If a block must output a whole frame at once, make sure the frame fits: `set_min_output_buffer()` on the block or in GRC's Advanced tab ("Min Output Buffer").	"Buffer too small for min_noutput_items", then a silent freeze (M2: OFDM packets ≥ 300 bytes).
4	Do not trust lengths that come from data (headers, tags). Check them against a maximum.	A corrupted or leaked length requests a huge output, causing the same freeze (M2: TX tag leaking into RX).
5	Watch which tags flow into your block. Use distinct tag keys or block tags (e.g. Channel Model `block_tags=True`).	Another block's metadata is misread as yours.
6	Return the true number of produced items. For vector ports, count vectors, not scalars.	Wrong data, or downstream waits forever.

Debugging checklist when a flowgraph "just freezes"

1. Read the terminal: look for `thread_body_wrapper :error` or "Buffer too small".
2. Remove blocks one at a time until the freeze disappears.
3. Raise min output buffers. If the freeze disappears, it is rule 3 or 4.
4. Put a Tag Debug block before the suspect block to see which tags arrive.
5. Run with a timeout (see `run_guarded()` in our `ofdm_packet_loss.py`) so tests report HANG instead of blocking forever.

7. When is a block an upstream contribution?

Our Gain block is a **learning exercise, not a contribution**: GNU Radio already has `blocks.multiply_const_ff` (we used it as the test reference). Before writing any block for upstream:

1. **Search** GNU Radio and CGRAN (the catalogue of OOT modules) for existing functionality.
2. **Ask** in an issue or in chat whether maintainers want it in-tree or as an OOT module. Most new blocks start as OOT modules, which is exactly how all three GSoC 2026 projects were delivered.
3. If it belongs in-tree: C++ implementation, Python bindings, GRC yml, QA tests, a documentation comment in the public header, and an example. Format C++ with `clang-format` (version 14, see `tools/clang_format.py`) and Python with PEP8 (`tools/check_pep8.sh`).

Contributions that are often more valuable than a new block: **tests** for existing blocks (our M2 contribution), **bug fixes**, **examples** and **documentation**.

7.1 The pull-request mechanics

```
# once
git clone https://github.com/<you>/gnuradio.git        # your fork
cd gnuradio && git remote add upstream https://github.com/gnuradio/gnuradio.git
# per contribution
git fetch upstream && git checkout -b ofdm-multipacket-test upstream/main
# ... edit, build, run ctest -R qa_ofdm_txrx ...
git commit -s                 # message: "digital: ofdm: add back-to-back multi-packet
loopback test"
git push origin ofdm-multipacket-test
# open the PR on GitHub, paste PR_DESCRIPTION.md, link the issue ("Refs #6891")
```

Commit message rules (GREP1): subject starts with the component (`digital:`, `runtime:`), lines under 72 characters, and the body explains why.

8. Exercises (in order of difficulty)

1. Change the Gain block to complex input/output (`np.complex64`). Which test needs to change?
2. Write a `decim_block` that averages every N samples. Test it with N that does not divide the input length.
3. Write a general block that drops every sample whose magnitude is below a threshold. Implement `forecast()` and `consume_each()` correctly, and write a test with 100 000 samples.
4. Break rule 3 on purpose: make your block call `set_min_noutput_items(100000)`, observe the freeze, then fix it with `set_min_output_buffer()`. Write down what you saw.
5. Build the C++ `gain_ff` from §5 and make its C++ QA test pass.

References

- GNU Radio wiki tutorials: "Creating Your First Block", "Creating C++ OOT with gr-modtool", "Creating Python OOT with gr-modtool", "Types of Blocks", "Stream Tags", "Tagged Stream Blocks".

- GNU Radio source: `gnuradio-runtime/include/gnuradio/block.h`, `gnuradio-runtime/lib/block_executor.cc`, `gr-digital/lib/header_payload_demux_impl.cc`.
- GNU Radio CONTRIBUTING.md and GREP1 (coding guidelines).
- Team G1 M2 repository: `Code/gain_block_tutorial/`, `Code/experiments/freeze_buffer_test.py`.